

分库分表如何实现平滑上线&回滚

需求背景

当数据库访问压力或者单表数据过高时，需要考虑进行分库分表或者使用分布式数据库扩展查询和存储能力，这里以 ShardingSphere 分库分表举例。

1. 什么时候分库

分库主要解决的是并发量大的问题。

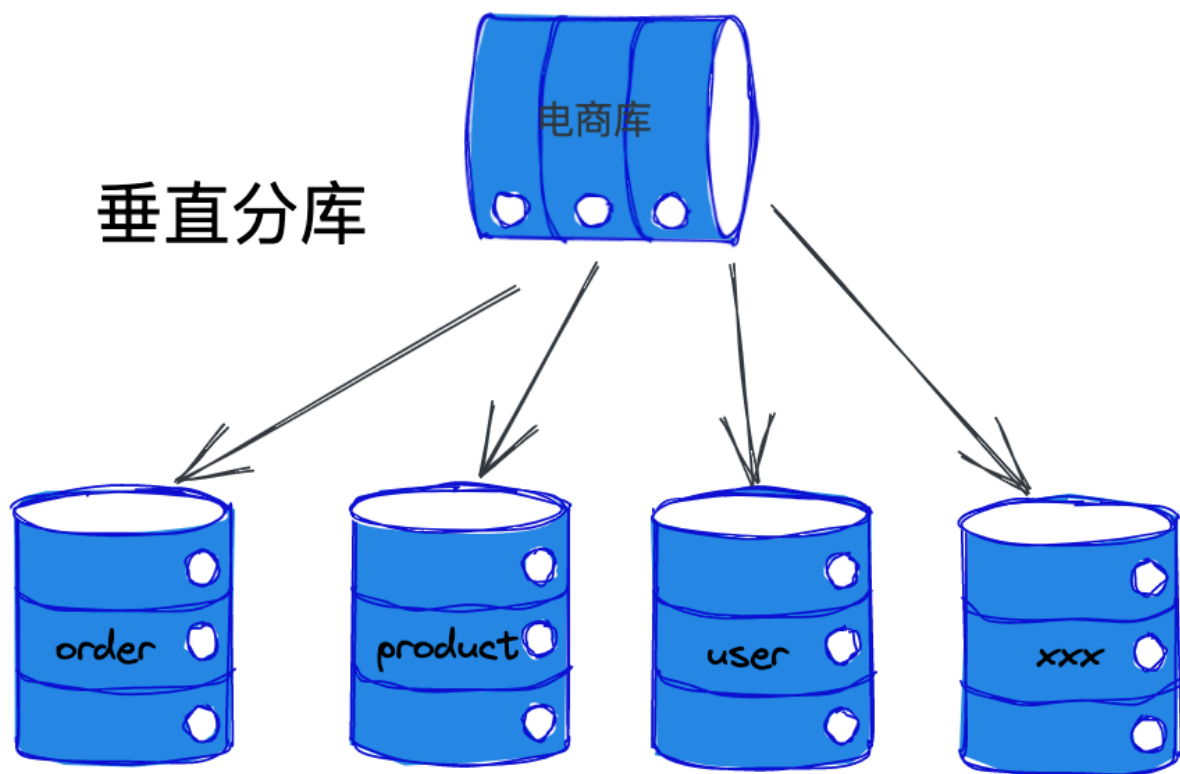
因为并发量一旦上来了，那么数据库就可能会成为瓶颈，因为数据库的连接数是有限的，虽然可以调整，但是也不是无限调整的。

当你的数据库的读或者写的 QPS 过高，导致你的数据库连接数不足了的时候，就需要考虑分库了，通过增加数据库实例的方式来提供更多的可用数据库链接，从而提升系统的并发度。

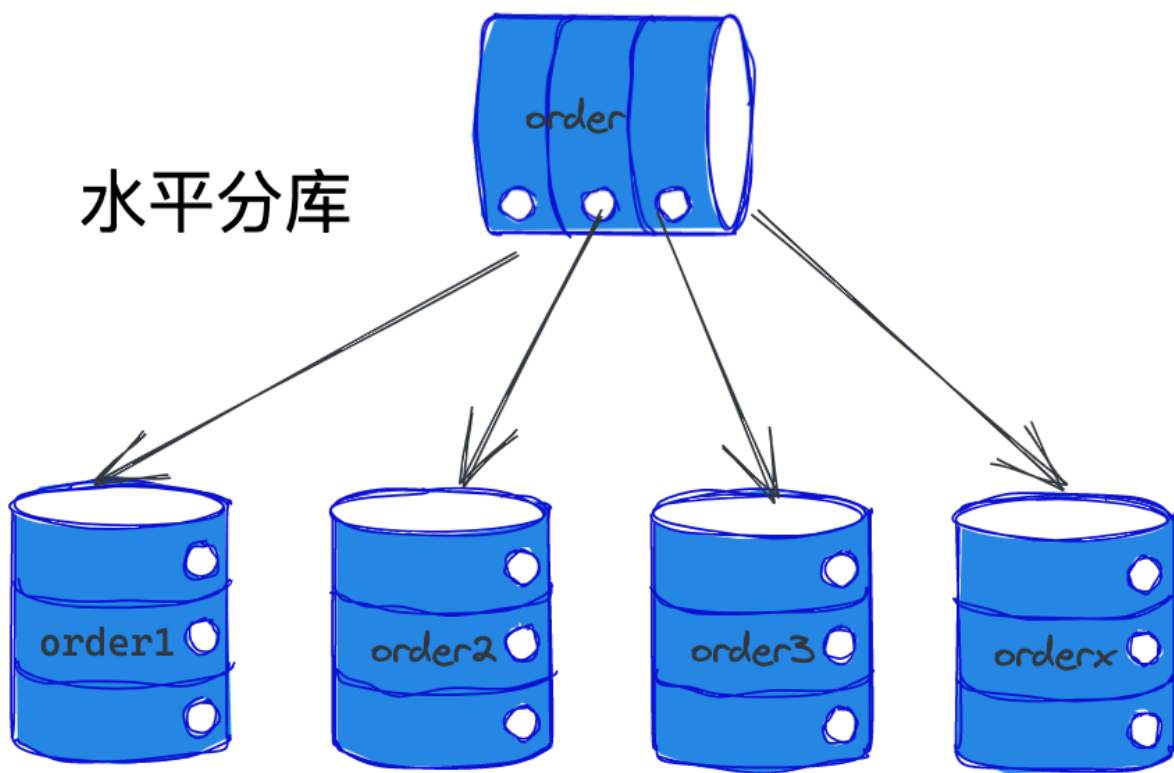
一般来说，如果是单体应用拆微服务，那么将原来的单体应用数据库拆成不同微服务的数据库。比如电商库拆分为：订单库、商品库、会员库以及物流库等。

如果已经是微服务方式拆分的库，那么就需要进行继续拆，比如订单库拆分为订单库0_64。

垂直分库



水平分库



2. 什么时候分表

分库主要解决的是并发量大的问题，那分表其实主要解决的是数据量大的问题。

假如你的单表数据量非常大，因为并发不高，数据量连接可能还够，但是存储和查询的性能遇到了瓶颈了，你做了很多优化之后还是无法提升效率的时候，就需要考虑做分表了。



3. 平滑上线 & 快速回滚

对于分库分表，最为稳妥的方式就是服务停止一段时间，比如晚上 12 点到凌晨 6 点。在这个时间段验证分库分表功能，以及排查是否有潜在问题。

但是，对于重要业务来说，这个停服是不可接受的。所以，做分库分表之前，需要提前考虑两点：

- 1. 如何保证在进行切换分片表查询的时候，对业务没有影响？
- 2. 数据库分库分表上线后，出现问题如何快速回滚？

环境初始化

假设，目前有支付信息表 `pay\_info`，按照年进行分片，拆分出 2022 年到 2026 年。

name	schema	kind	charset	collation	engine
 pay_info	pay_man...	TABLE	utf8mb4	utf8mb4_ge...	InnoDB
 pay_info_shard_2022	pay_man...	TABLE	utf8mb4	utf8mb4_ge...	InnoDB
 pay_info_shard_2023	pay_man...	TABLE	utf8mb4	utf8mb4_ge...	InnoDB
 pay_info_shard_2024	pay_man...	TABLE	utf8mb4	utf8mb4_ge...	InnoDB
 pay_info_shard_2025	pay_man...	TABLE	utf8mb4	utf8mb4_ge...	InnoDB

MySQL 创建支付测试数据库 `pay\_manager\_test`。

创建上述数据库表 pay_info, 分片表 pay_info_shard_xxxx 同支付表结构一致, 仅表名不同。

```
CREATE TABLE `pay_info_shard_2022` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'ID',  
  `pay_no` varchar(128) DEFAULT NULL COMMENT '支付单号',  
  `pay_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '支付时间',  
  `create_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
  `update_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '修改时间',  
  `del_flag` tinyint(1) DEFAULT '0' COMMENT '删除标志',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `id` (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=1599950067182321665 DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE `pay_info_shard_2023` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'ID',  
  `pay_no` varchar(128) DEFAULT NULL COMMENT '支付单号',  
  `pay_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '支付时间',  
  `create_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
  `update_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '修改时间',  
  `del_flag` tinyint(1) DEFAULT '0' COMMENT '删除标志',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `id` (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=1599950067182321665 DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE `pay_info_shard_2024` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'ID',  
  `pay_no` varchar(128) DEFAULT NULL COMMENT '支付单号',  
  `pay_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '支付时间',  
  `create_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',  
  `update_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '修改时间',  
  `del_flag` tinyint(1) DEFAULT '0' COMMENT '删除标志',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `id` (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=1599950067182321665 DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE `pay_info_shard_2025` (  
  `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT 'ID',  
  `pay_no` varchar(128) DEFAULT NULL COMMENT '支付单号',  
  `pay_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '支付时间',  
  `create_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
```

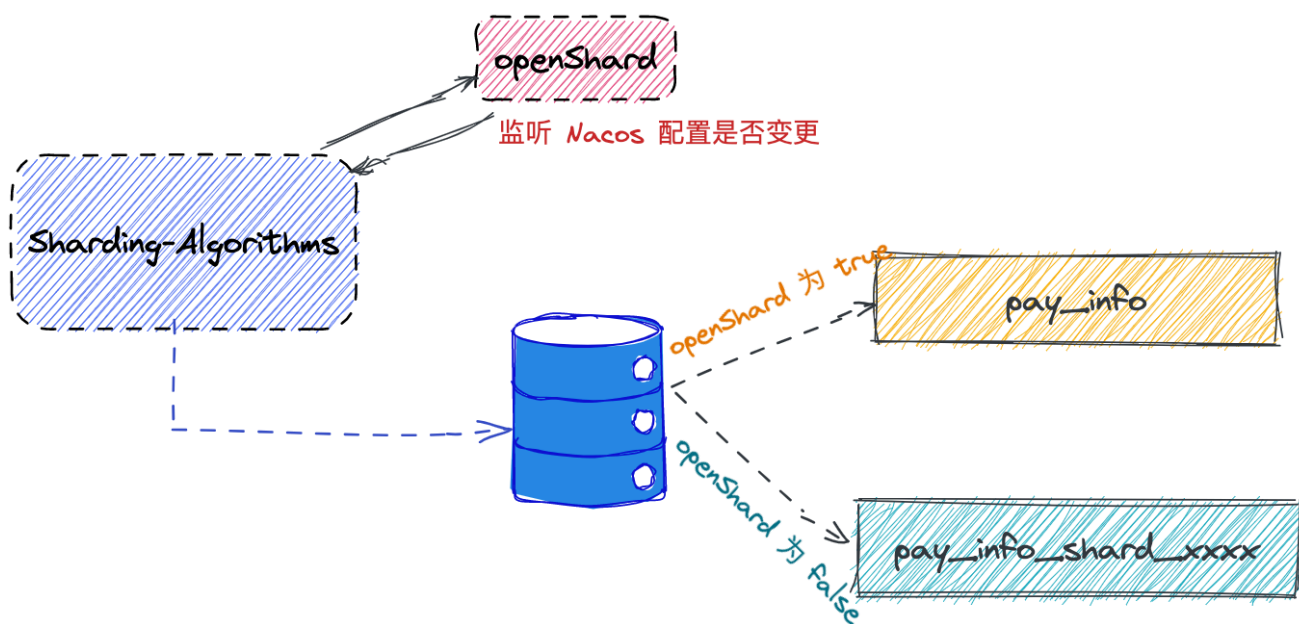
```
`update_time` datetime DEFAULT CURRENT_TIMESTAMP COMMENT '修改时间',  
`del_flag` tinyint(1) DEFAULT '0' COMMENT '删除标志',  
PRIMARY KEY (`id`),  
UNIQUE KEY `id` (`id`)  
) ENGINE=InnoDB AUTO_INCREMENT=1599950067182321665 DEFAULT CHARSET=utf8mb4;
```

分库分表平滑上线以及快速回滚需要依赖 Canal、Nacos 中间件，如想测试效果请进行安装。

平滑上线

1. 平滑方案

理想情况，先通过 ShardingSphere-JDBC 方式改造持久层查询代码，上线后依旧查询 pay_info 表，通过在 Nacos 配置中心放置个动态分片配置键，通过该配置变更决定查询旧表或者分片表。



2. ShardingSphere 分片

Maven 引入 ShardingSphere-JDBC Jar。

```
<dependency>  
  <groupId>org.apache.shardingsphere</groupId>  
  <artifactId>shardingsphere-jdbc-core-spring-boot-starter</artifactId>
```

```
<version>5.2.0</version>
</dependency>
```

通过实现 ShardingSphere 扩展接口 StandardShardingAlgorithm 自定义分片算法:

```
import com.google.common.collect.Lists;
import org.apache.shardingsphere.sharding.api.sharding.standard.PreciseShardingValue;
import org.apache.shardingsphere.sharding.api.sharding.standard.RangeShardingValue;
import
org.apache.shardingsphere.sharding.api.sharding.standard.StandardShardingAlgorithm;
import org.opengoofy.congomall.springboot.starter.base.ApplicationContextHolder;
import org.opengoofy.congomall.test.smooth.sharding.config.ShardRollbackProperties;
import org.slf4j.MDC;

import java.util.Collection;
import java.util.Date;
import java.util.Properties;

/**
 * 支付创建时间分片算法
 */
public class PayCreateTimeStandardShardingAlgorithm implements
StandardShardingAlgorithm<Date> {

    @Override
    public String doSharding(Collection<String> availableTargetNames,
PreciseShardingValue<Date> shardingValue) {
        if (!getOpenShard()) {
            return "pay_info";
        }
        return "pay_info_shard_2022";
    }

    @Override
    public Collection<String> doSharding(Collection<String> availableTargetNames,
RangeShardingValue<Date> shardingValue) {
        if (!getOpenShard()) {
            return Lists.newArrayList("pay_info");
        }
        return Lists.newArrayList("pay_info_shard_2022", "pay_info_shard_2023",
"pay_info_shard_2024", "pay_info_shard_2025");
    }
}
```

```

    }

    @Override
    public Properties getProps() {
        return null;
    }

    @Override
    public void init(Properties properties) {

    }

    private boolean getOpenShard() {
        ShardRollbackProperties rollbackProperties =
        ApplicationContextHolder.getBean(ShardRollbackProperties.class);
        return rollbackProperties.getOpenShard();
    }
}

```

这里自定义分片算法只是为了演示平滑上线，不具备线上运行能力。

ShardingSphere 按支付表进行分片配置：

```

spring.shardingsphere.datasource.ds-0.type=com.zaxxer.hikari.HikariDataSource
spring.shardingsphere.datasource.ds-0.driver-class-name=com.mysql.jdbc.Driver
spring.shardingsphere.datasource.ds-0.jdbc-
url=jdbc:mysql://127.0.0.1:3306/pay_center_test?characterEncoding=utf-
8&zeroDateTimeBehavior=convertToNull&transformedBitIsBoolean=true&serverTimezone=GMT%2B8
spring.shardingsphere.datasource.ds-0.username=root
spring.shardingsphere.datasource.ds-0.password=root

# 分片数据库别名
spring.shardingsphere.datasource.names=ds-0
# 是否打印 ShardingSphere SQL 语句
spring.shardingsphere.props.sql-show=true
# 分片物理表真实节点
spring.shardingsphere.rules.sharding.tables.pay_info.actual-data-nodes=ds-
0.pay_info_shard_${2022..2025},ds-0.pay_info
# 分片算法名称
spring.shardingsphere.rules.sharding.tables.pay_info.table-strategy.standard.sharding-
algorithm-name=sharding_by_time

```

```

# 分片策略之分键定义
spring.shardingsphere.rules.sharding.tables.pay_info.table-strategy.standard.sharding-
column=create_time
# 分片算法策略类型
spring.shardingsphere.rules.sharding.sharding-
algorithms.sharding_by_time.props.strategy=standard
# 分片算法实现类型
spring.shardingsphere.rules.sharding.sharding-
algorithms.sharding_by_time.type=CLASS_BASED
# 分片算法实现类
spring.shardingsphere.rules.sharding.sharding-
algorithms.sharding_by_time.props.algorithmClassName=org.opengoofy.congomall.test.smooth.
sharding.shard.PayCreateTimeStandardShardingAlgorithm

```

这里注意 `spring.shardingsphere.rules.sharding.tables.pay_info.actual-data-nodes`，除了 `ds-0.pay_info_shard_${2022..2025}` 外还需要加上 `ds-0.pay_info`，不然分片时会报错找不到分片表。

3. Nacos 新增动态配置

Nacos 配置中心增加一个配置：openShard=false，代码中增加运行时动态变更配置类，负责读取最新的 openShard。

```

import lombok.Data;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.cloud.context.config.annotation.RefreshScope;
import org.springframework.stereotype.Component;

/**
 * 分片回滚配置类
 */
@Component
@RefreshScope
@Data
public class ShardRollbackProperties {

    @Value("${openShard:false}")
    private Boolean openShard;
}

```

准备工作完成，即可通过配置中心 openShard 控制是否查询普通表或者分片表，两者切换不需要重启服务。

通过上述流程代码，已完成分片的平滑上线。但是还有一个问题，出现问题如何回滚？

快速回滚

1. 回滚方案

如果分库分表运行时发现存在业务问题，那么需要紧急回滚，再切换回查询主表。

回滚需要两个条件：

- 关闭分片功能：如果需要快速回滚，将 Nacos 动态配置 openShard 开关修改为 false；
- 双写：双写的逻辑就是在写入分片表后再将数据写入旧表 pay_info。

关闭分片功能在平滑上线部分已经完成，剩下的就是将数据进行双写了。

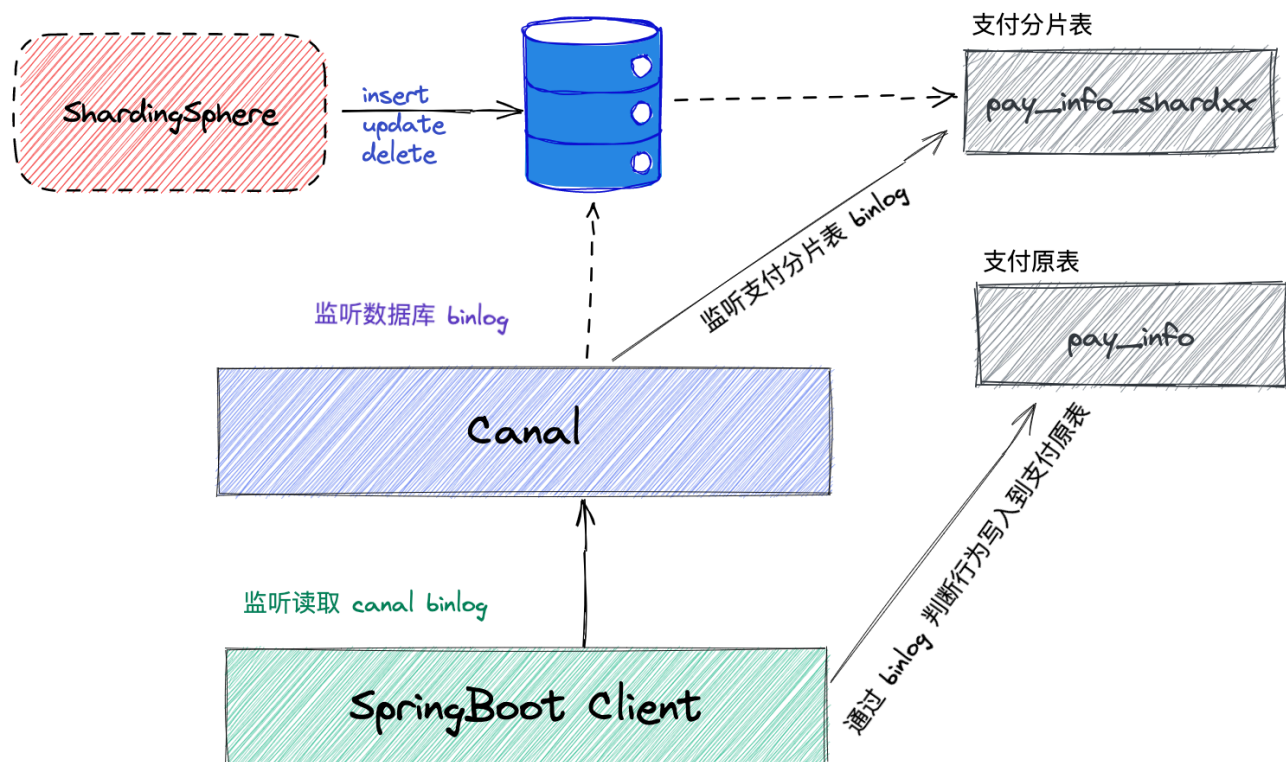
2. 数据双写

双写方案如下：

1. 代码中进行操作，写完分片表再写入旧表。缺点是需要改动代码，工作量繁琐，容易有遗漏。
2. 通过切面或者 MyBatis 插件判断，写入分片表后再写入旧表。优点是不需要改动代码。缺点是不支持事务并且需要添加额外数据源。
3. 通过数据同步组件 Canal 解析 Binlog 异步写入。优点是历史代码无影响，并且不需要添加额外数据源。

通过比较，使用 Canal 当作双写支持方案。具体方案为：Canal 解析到 Binlog 后投递 MQ，客户端从 MQ 直接消费。

需要注意的是，使用 MQ 顺序消息，避免出现 ABA 问题。



Canal 监听中，需要通过正则表达式监听表信息：pay_center\\.pay_info_shard\\.* 这种方式会监听 pay_info_shard_ 开头的表。

3. 改造分片算法

最后，有一个关于双写数据同步的问题需要解决。

那就是，如果我们在支付原应用中双写旧表，会遇到一个问题，ShardingSphere 会把 pay_info 解析为支付分片表，这个问题有两种解决方案：

1. 双写旧表通过动态数据源单独创建个数据源，从该数据源获取数据库连接并操作旧 pay_info 表。
2. 在支付分片算法中，加入双写标签。如果发现是双写请求，那么返回旧表。

从实现复杂度上来说，第二种是最轻量以及容易理解的。仅需要改造分片算法，示例如下：

```
import com.google.common.collect.Lists;
import org.apache.shardingsphere.sharding.api.sharding.standard.PreciseShardingValue;
import org.apache.shardingsphere.sharding.api.sharding.standard.RangeShardingValue;
import
```

```

org.apache.shardingsphere.sharding.api.sharding.standard.StandardShardingAlgorithm;
import org.opengoofy.congomall.springboot.starter.base.ApplicationContextHolder;
import org.opengoofy.congomall.test.smooth.sharding.config.ShardRollbackProperties;
import org.slf4j.MDC;

import java.util.Collection;
import java.util.Date;
import java.util.Properties;

/**
 * 支付创建时间分片算法
 */
public class PayCreateTimeStandardShardingAlgorithm implements
StandardShardingAlgorithm<Date> {

    @Override
    public String doSharding(Collection<String> availableTargetNames,
PreciseShardingValue<Date> shardingValue) {
        if (!getOpenShard() || shardSyncData()) {
            return "pay_info";
        }
        return "pay_info_shard_2022";
    }

    @Override
    public Collection<String> doSharding(Collection<String> availableTargetNames,
RangeShardingValue<Date> shardingValue) {
        if (!getOpenShard() || shardSyncData()) {
            return Lists.newArrayList("pay_info");
        }
        return Lists.newArrayList("pay_info_shard_2022", "pay_info_shard_2023",
"pay_info_shard_2024", "pay_info_shard_2025");
    }

    @Override
    public Properties getProps() {
        return null;
    }

    @Override
    public void init(Properties properties) {

```

```

    }

    private boolean getOpenShard() {
        ShardRollbackProperties rollbackProperties =
ApplicationContextHolder.getBean(ShardRollbackProperties.class);
        return rollbackProperties.getOpenShard();
    }

    private boolean shardSyncData() {
        return MDC.get("shard_sync") == null ? false : true;
    }
}

```

增加 shardSyncData 方法，通过 MDC 来传递双写标识。

监听 Canal 的消费端，操作数据库前，向 MDC 设置一个 shard_sync Key 即可，操作完数据库删除该 Key。

伪代码如下：

```

// 接收到 Canal 变更数据
try {
    MDC.put("shard_sync", "true");
    // 操作数据库...
} finally {
    MDC.remove("shard_sync");
}

```

4. Canal 同步逻辑

Canal 消费同步示例代码：

```

import cn.hutool.core.util.IdUtil;
import com.alibaba.otter.canal.client.CanalConnector;
import com.alibaba.otter.canal.client.CanalConnectors;
import com.alibaba.otter.canal.common.utils.AddressUtils;
import com.alibaba.otter.canal.protocol.CanalEntry.*;
import com.alibaba.otter.canal.protocol.Message;

```

```

import lombok.extern.slf4j.Slf4j;
import org.opengoofy.congomall.test.smooth.sharding.dao.entity.PayDO;
import org.opengoofy.congomall.test.smooth.sharding.dao.mapper.PayMapper;
import org.slf4j.MDC;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

import javax.annotation.Resource;
import java.net.InetSocketAddress;
import java.util.Date;
import java.util.List;
import java.util.Objects;

/**
 * 支付 Canal 客户端
 */
@Slf4j
@Component
public class PayCanalClient implements CommandLineRunner {

    @Resource
    private PayMapper payMapper;

    @Override
    public void run(String... args) throws Exception {
        CanalConnector connector = CanalConnectors.newSingleConnector(new
InetSocketAddress(AddressUtils.getHostIp(),
        11111), "example", "", "");
        int batchSize = 1000;
        try {
            connector.connect();
            connector.subscribe("pay_center\\pay_info_shard.*");
            while (true) {
                Message message = connector.getWithoutAck(batchSize);
                long batchId = message.getId();
                int size = message.getEntries().size();
                if (batchId == -1 || size == 0) {
                    try {
                        Thread.sleep(1000);
                    } catch (InterruptedException e) {
                    }
                }
            }
        }
    }
}

```

```

        } else {
            printEntry(message.getEntries());
        }
        connector.ack(batchId);
    }
} finally {
    connector.disconnect();
}
}

private void printEntry(List<Entry> entrys) {
    MDC.put("shard_sync", "true");
    for (Entry entry : entrys) {
        if (entry.getEntryType() == EntryType.TRANSACTIONBEGIN ||
entry.getEntryType() == EntryType.TRANSACTIONEND) {
            continue;
        }
        RowChange rowChange;
        try {
            rowChange = RowChange.parseFrom(entry.getStoreValue());
        } catch (Exception e) {
            throw new RuntimeException("ERROR ## parser of error change-event has an
error, data: " + entry, e);
        }
        EventType eventType = rowChange.getEventType();
        System.out.println(String.format("=====> binlog[%s:%s],
name[%s,%s], eventType: %s",
            entry.getHeader().getLogfileName(),
entry.getHeader().getLogfileOffset(),
            entry.getHeader().getSchemaName(), entry.getHeader().getTableName(),
            eventType));
        for (RowData rowData : rowChange.getRowDatasList()) {
            if (eventType == EventType.DELETE) {
                printColumn(rowData.getBeforeColumnsList());
            } else if (eventType == EventType.INSERT) {
                printColumn(rowData.getAfterColumnsList());
                // 这里仅对新增操作演示
                insert(rowData);
            } else {
                System.out.println("-----> before");
                printColumn(rowData.getBeforeColumnsList());
            }
        }
    }
}

```

```

        System.out.println("-----> after");
        printColumn(rowData.getAfterColumnsList());
    }
}
}
MDC.remove("shard_sync");
}

private static void printColumn(List<Column> columns) {
    for (Column column : columns) {
        System.out.println(column.getName() + ": " + column.getValue() + "
update=" + column.getUpdated());
    }
}

private void insert(RowData rowData) {
    PayDO payDO = new PayDO();
    List<Column> afterColumnsList = rowData.getAfterColumnsList();
    for (Column column : afterColumnsList) {
        if (Objects.equals(column.getName(), "id")) {
            payDO.setId(Long.parseLong(column.getValue()));
        } else if (Objects.equals(column.getName(), "pay_no")) {
            payDO.setPayNo(column.getValue());
        }
    }
    payDO.setCreateTime(new Date());
    payDO.setId(IdUtil.getSnowflakeNextId());
    payMapper.insert(payDO);
}
}

```

代码详情查看：[congomall-test-all/congomall-smooth-sharding-test](#)

文末总结

文中通过渐进式的讲解帮助大家理解了什么是分库分表，以及分库或者分表如何选择，相信大家对这些的技能点已经比较熟悉了。

通过分库分表进而引出平滑上线和快速回滚的知识，上述方案已在我司执行过，帮助核心业务线顺利分库分表改造，期间未出现问题。

Q: 使用 Canal 进行双写，还需要通过 SpringBoot 编写代码，有没有更方便的逻辑？

A: 可以通过 StreamSets 或者 CloudCanal 等数据同步产品代替，无需代码编写。如果公司已
有相关产品，可以直接使用。没有的话，Canal 也是个不错的选择。

更新: 2024-04-14 13:43:06

原文: <<https://www.yuque.com/magestack/12306/apt0u4ql7c1m6098>>